

Somewhere, something went terribly wrong

INFO 641

Généralisation/Héritage

Généralisation / factorisation

- Problème: deux classes ont des propriétés « proches »
 - peut-on éviter de décrire/programmer deux fois les propriétés « identiques »?

- Exemples :

Livre
- titre : String - auteur : String - editeur : String - isbn : int
+ afficher() : void + toString() : String + getEditeur() : String ...

Rapport
- titre : String - auteur : String - org : String - numero : int
+ afficher() : void + toString() : String + setNumero() : int ...

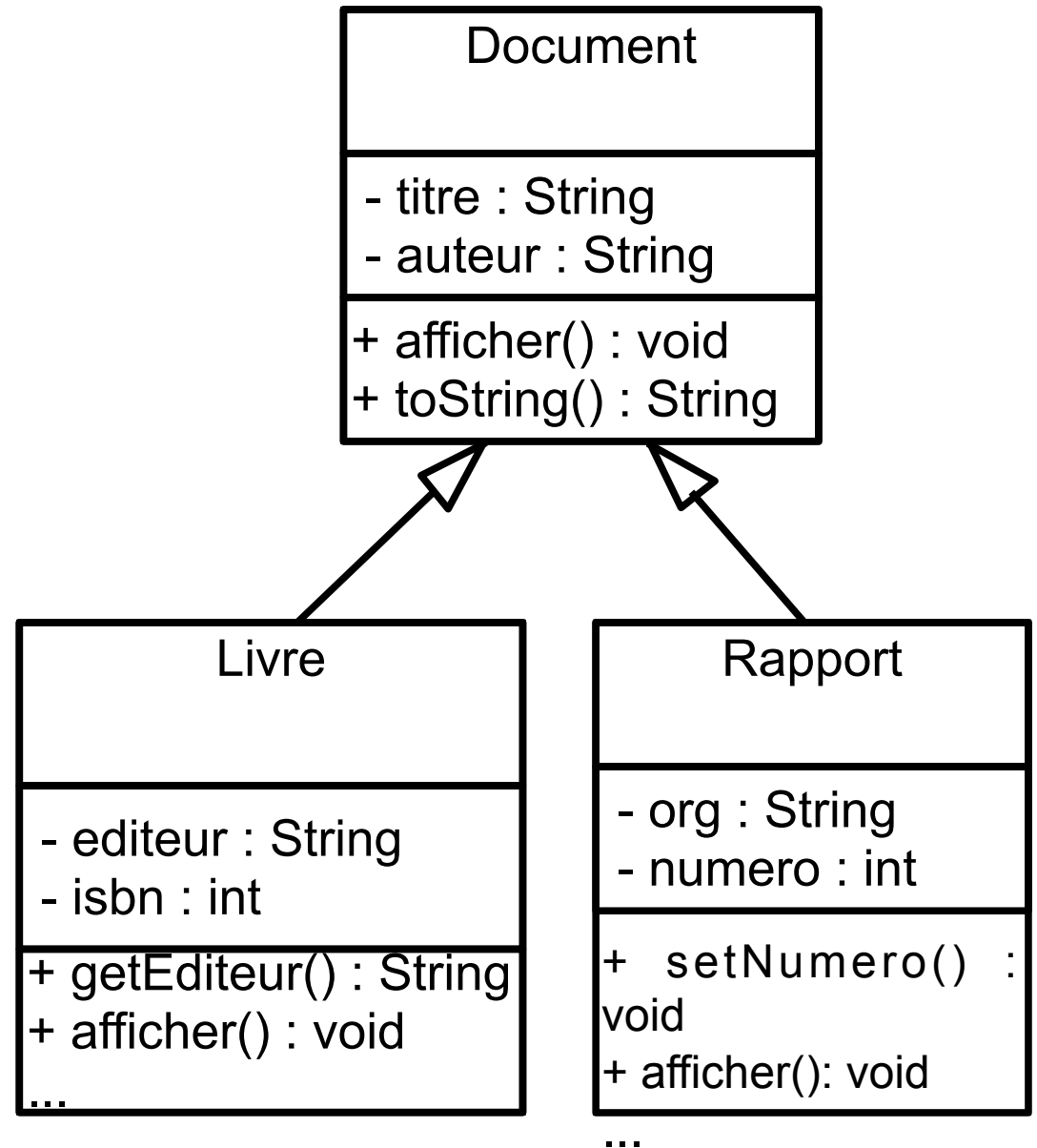
Héritage

généralisation / factorisation

- On définit une classe Document qui regroupe ces propriétés communes
- On indique le lien entre Document et Livre, Rapport:
 - en analyse/conception, on précise:
Document **généralise** Livre et Rapport
 - dans les langages à objets, on déclare:
Livre, Rapport **héritent** de Document
- Deux points de vue pour la même notion

Héritage - notation UML

- Flèche avec triangle creux
- Les propriétés héritées ne sont pas indiquées dans la classe fille (en général...)



Héritage comme généralisation

- Document **généralise** Livre et Rapport (A/C OO)
 - Document est une **super-classe** (**classe mère**) de Livre et Rapport
- Livre et Rapport **héritent** de Document (A/C/P OO)
 - Livre et Rapport sont des **sous-classes** (**classes filles**) de Document

Héritage

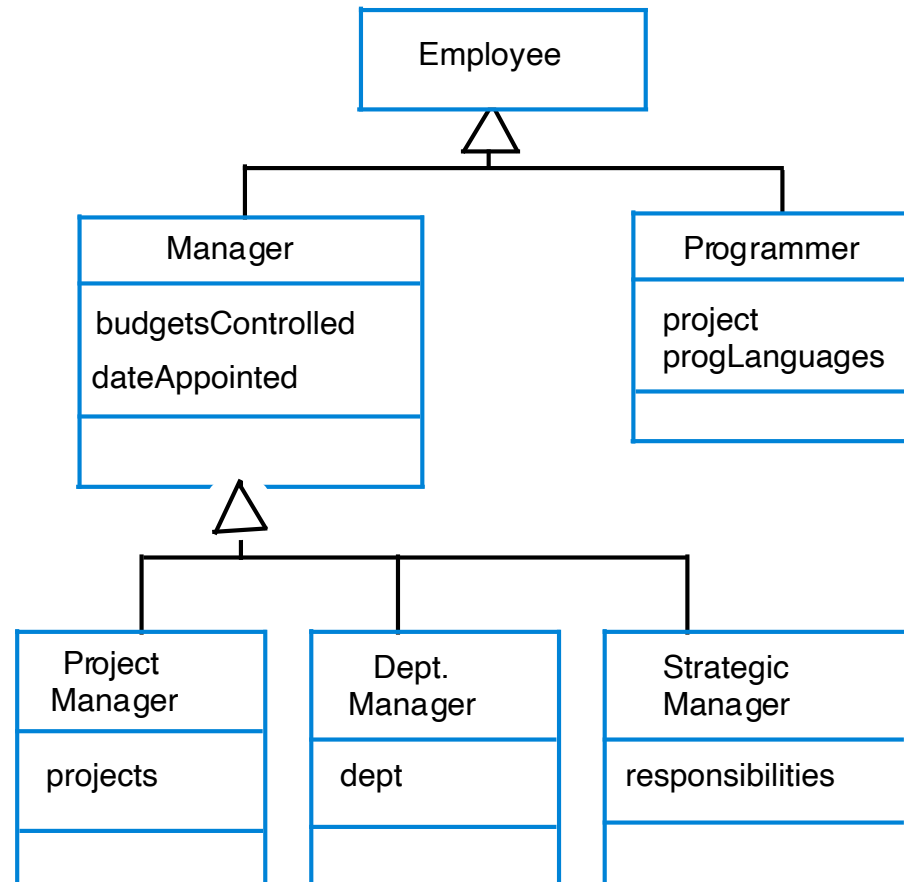
restriction / extension

- La classe B hérite de la classe A:
B est **une restriction et une extension** de A
- **Restriction**: toute instance de B est aussi une instance de A:
 - $\text{Instances}(B) \subseteq \text{Instances}(A)$
 - Les Livres sont des Documents
- **Extension**: les instances de B ont plus de fonctionnalités que celles de A:
 - Plus d'attributs et/ou plus d'opérations ou des opérations plus fines
 - editeurs, isbn, getEditeur(), afficher()...

Exercice

- Donnez un exemple de hierarchie avec une super-classe et deux sous-classes

Hiérarchie d'héritage



- La relation d'héritage définit un graphe (non connexe en général) dont les noeuds sont les classes

Héritages – aspects langages

écriture de la classe

- Selon les langages, un attribut **private** est inaccessible ou pas dans la sous-classe
- En Java, utilisez le qualificateur **protected**:

```
Public class Document
{
    protected String titre;
    private String auteur;
    ...
}
```

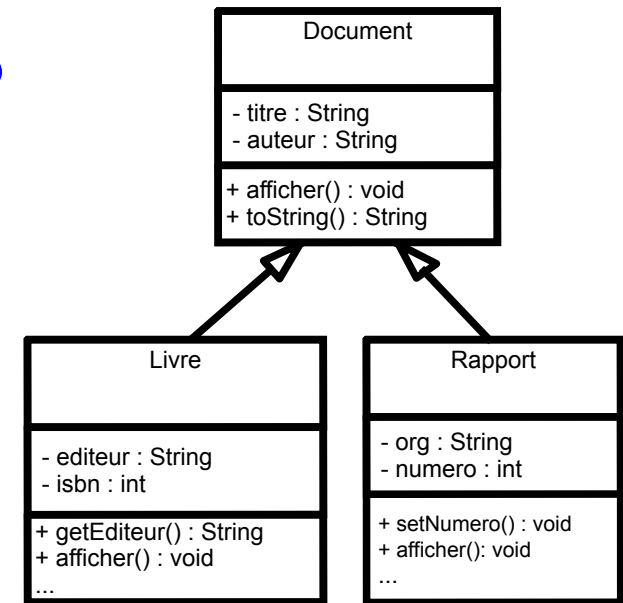
- **protected** rend les attributs visibles dans les sous-classes seulement

Héritage - aspects langages

utilisation des attributs

- Les attributs de la super-classe s'utilisent comme les attributs de la classe (ce sont des attributs de la sous-classe)

```
Public class Livre{
```



```
...
```

```
Public void printInfo(){
    System.out.println(titre);
    System.out.println(isbn);
    System.out.println(auteur);
```

OK or not OK?

```
}
```

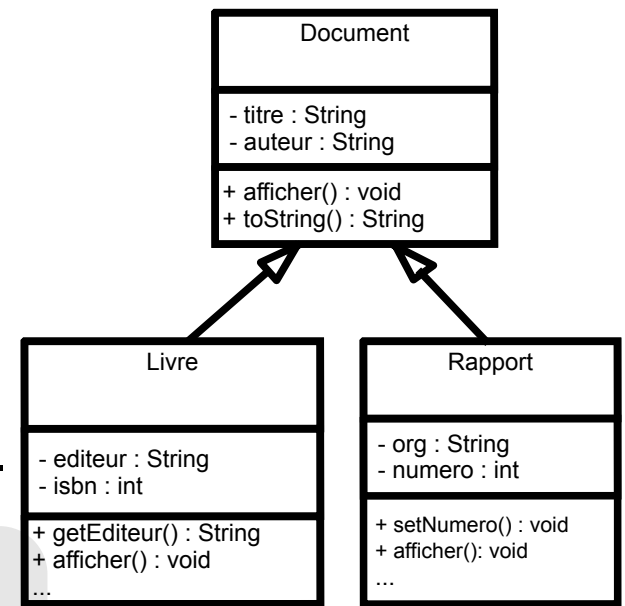
- Évitez des noms de champs identiques dans les deux classes, sinon utilisez this, self, current, etc.

Classes et types

- Que se passe-t-il lorsque l'on « mélange » des variables et objets de classes et sous-classes?

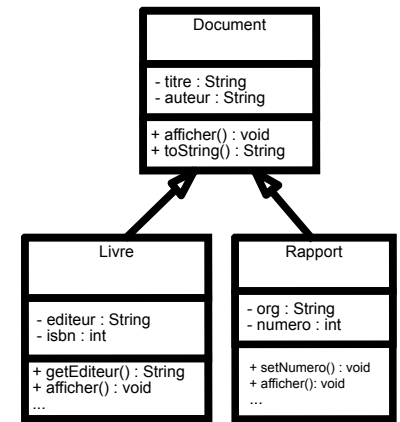
- ```
Document d1;
Livre l1 = new Livre(...); → l1 est de
type Livre
Rapport r1 = new Rapport(...); → r1 est de
type de Rapport
...
d1 = l1; → d1 est de type Document
...
d1 = r1; → d1 est de type Document
...
```

- **Le type d'un objet de sous-classe est un sous-type du type des objets de la classe**



# Classes et types - prudence!

## langage objet à typage statique



- Que se passe-t-il lorsque l'on « mélange » des variables et objets de classes et sous-classes?
- Dans un langage à typage statique, attention!

```
String s = l1.getEditeur(); → OK
d1 = l1; → d1 est de type Document
s = d1.getEditeur(); → KO par le compilateur !
String a = d1.getAuteur(); → OK!
```

- Les propriétés d'une variable sont celles de son type déclaré, appelé le **type statique**

# Classes et types

## type statique et type dynamique

Une variable de type A, peut référencer tout objet d'une classe qui hérite de A (**polymorphisme**)

C hérite de B, qui hérite de A:

```
A x;
A a1 = new A(...);
B b1 = new B(...);
C c1 = new C(...);
x = a1; → OK
x = b1; → OK
x = c1 → OK
```

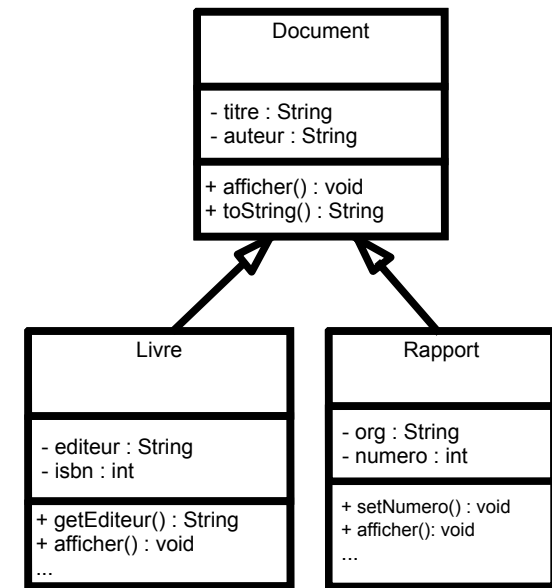
```
Public class A{
...}

Public class B extends A{
...}

Public class C extends B{
...}
```

- Le **type dynamique** d'une variable est le type de l'objet référencé par cette variable à un instant donné

# Héritage et opération



- Que devient la méthode `afficher()` ?

```
Livre l1; Rapport r1; Document d1;
...
d1 = l1;
...
l1.afficher(); → afficher() de Livre
r1.afficher(); → afficher() de Rapport
d1.afficher(); → afficher() de Livre
```

- **Nécessité de pouvoir afficher un document**

# Liaison dynamique

- À l'exécution, le système d'exécution (JVM en Java) choisit la version de la méthode **correspondant au type de l'objet à l'instant donné**:

```
• Document d1 = new Document(...);
 Rapport r1 = new Rapport(...);
 d1.afficher(); → afficher() de Document
 d1 = r1;
 d1.afficher(); → afficher() de Rapport
```

- Ce mécanisme s'appelle **la liaison dynamique** ("late binding")

# Premier bilan de l'héritage

- Héritage pour généraliser
- Hiérarchie de classes
- Polymorphisme
- Liaison dynamique

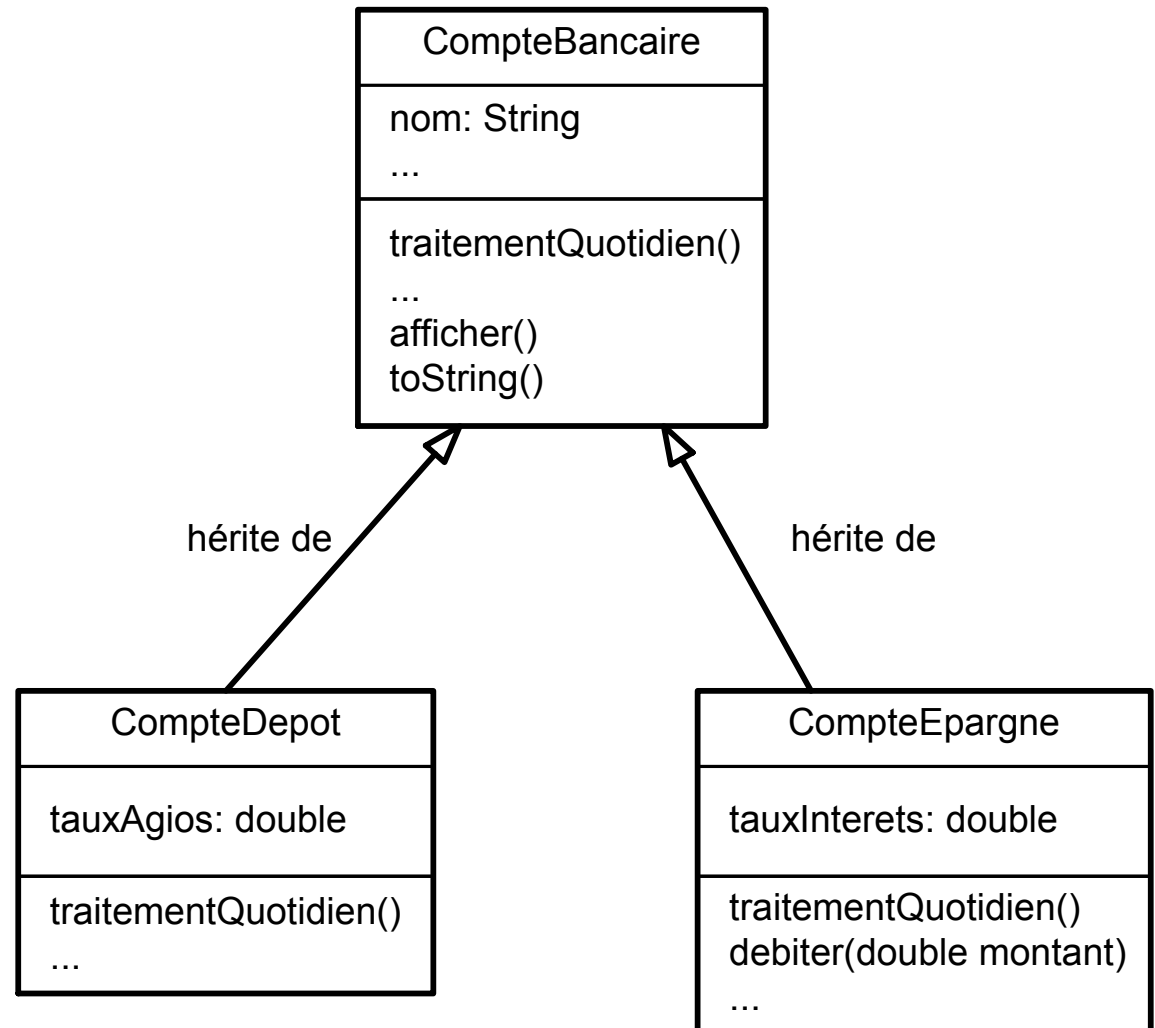


# Spécialisation de classe

- CompteBancaire:  
on dispose d'une procédure  
**traitementQuotidien()**  
qui met à jour le compte chaque nuit.
- On veut différencier deux types de comptes:
  - CompteDepot: agios si solde négatif (tauxAgios)
  - CompteEpargne: jamais de solde négatif,  
intérêts versés (tauxInterets)

# CompteBancaire et dérivés

- Les classes  
CompteDepot  
CompteEpargne  
spécialisent  
CompteBancaire



# Polymorphisme et liaison dynamique, encore...

- Tous les soirs: mettre à jour un tableau de Comptes bancaires de tout type:

```
ArrayList<CompteBancaire> comptes = new ArrayList<CompteBancaire>();
...
for (int i=0; i<comptes.size(); i++) {
 comptes.get(i).traitementQuotidien();
}
```

OU

```
for (CompteBancaire cb:comptes) {
 cb.traitementQuotidien();
}
```

- Même code quelque soit la nature du compte

# Contrats et héritage invariants

- Une sous-classe B doit respecter le contrat de sa super-classe A
- Donc, tous les invariants de la super-classe A doivent être vérifiés dans la sous-classe B  
(il peut y en avoir d'autres spécifiques à B)
- Exemples:
  - CompteEpargne ajoute l'invariant "Solde positive"
  - Toutes les sous classes de CompteEpargne doivent respecter l'invariant "Solde positive"
  - CompteBancaire et CompteDepot ne sont pas contraint par cet invariant

# Contrats et héritage

## préconditions d'opération

- Soit  $op$  une méthode de **A** redéfinie dans **B**
  - Exemple: `traitementQuotien()` est redéfinie dans `CompteEpargne`
- En raison de la liaison dynamique, un appel de  $op$  sur  $A$ , peut en fait appeler  $op$  d'un objet  $B$ :

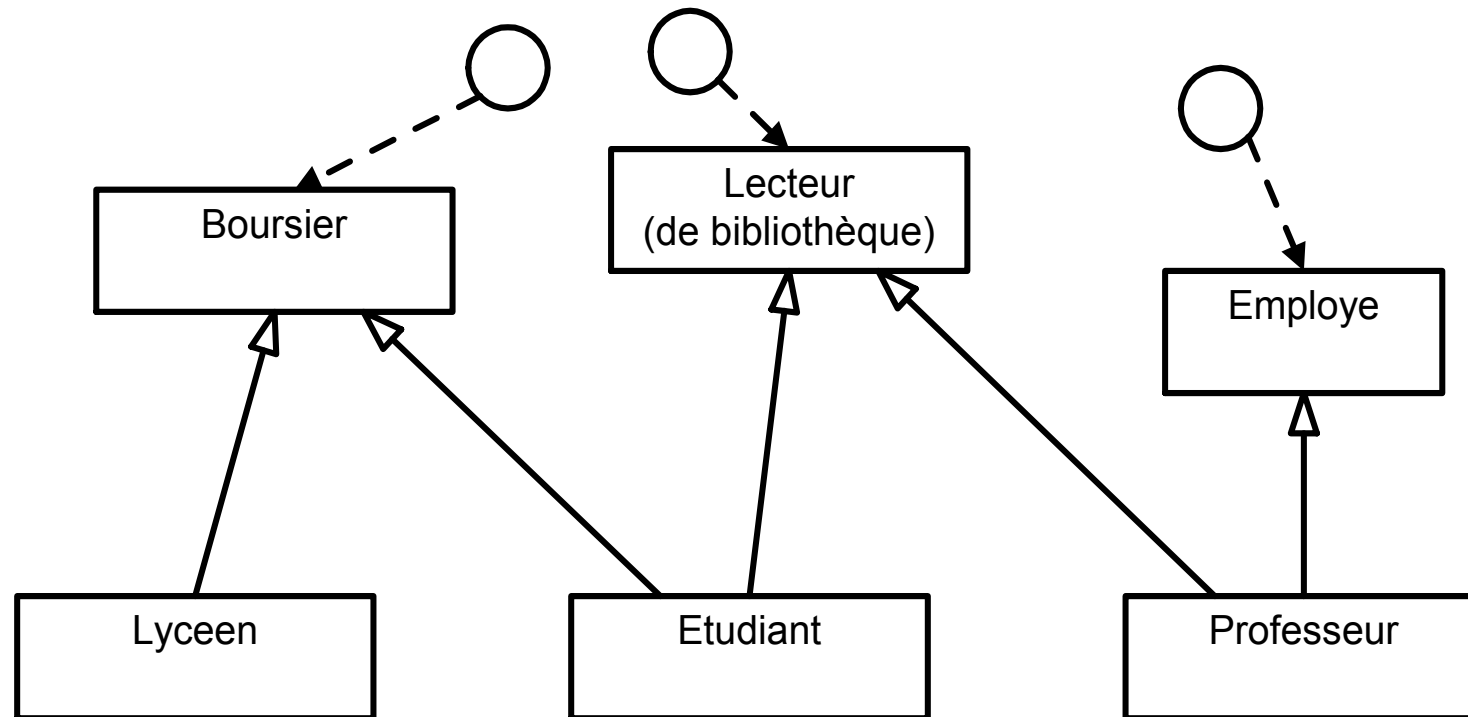
```
CompteBancaire cb = new CompteEpargne(...);
cb.traitementQuotidien(); → méthode de CompteEpargne
```

si les préconditions de  $op_A$  sont satisfaites, celles de  $B$  doivent être satisfaites automatiquement (pour permettre l'appel):

$$\text{pré}(op_A) \implies \text{pré}(op_B)$$

les  $\text{pré}(op_B)$  sont plus faibles que les  $\text{pré}(op_A)$

# Héritage multiple



Etudiant doit se comporter comme un Boursier et comme un Lecteur pour les gestionnaires de bourse ou de bibliothèque

- **Pas d'héritage multiple en Java, Smalltalk**